

Lab 03 Dopo

Herencia e interfaces ADEMAS Java desde consola

PRESENTADO POR:

Juanita Rubiano Fernandez

Cristian Santiago Moreno Ruiz

PRESENTADO A:

Maria Irma Diaz Roza

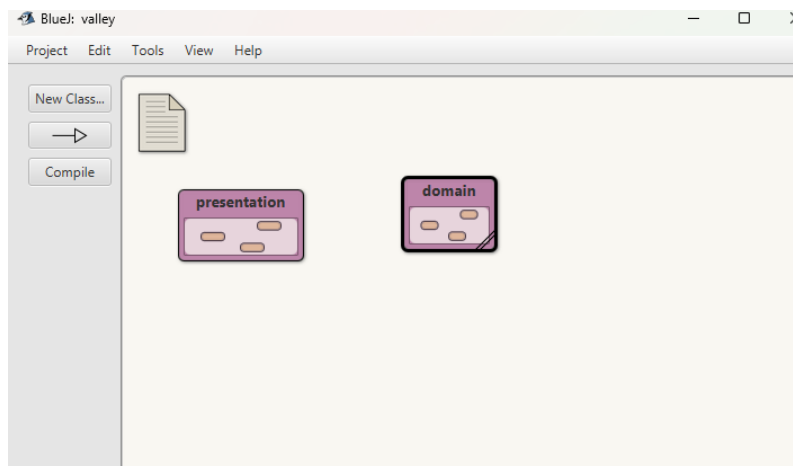
Universidad Escuela Colombiana De Ingeniería Julio Garavito

Bogotá D.C

2025 - 1

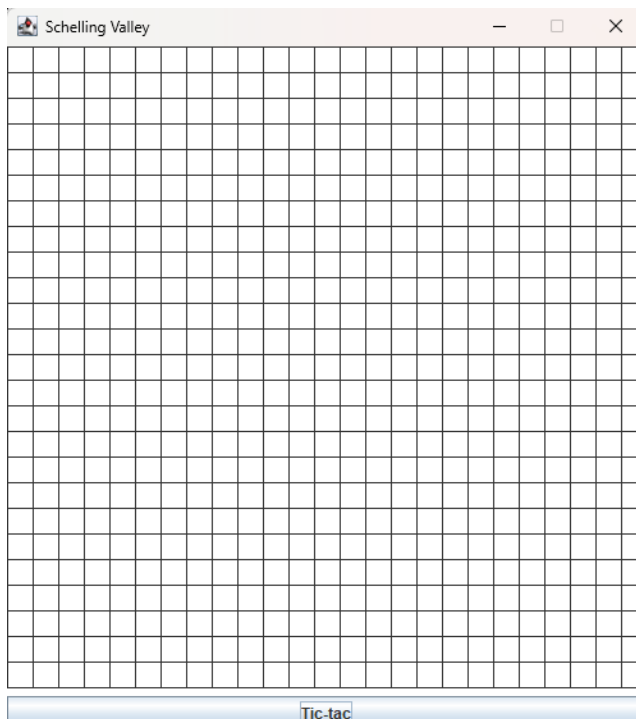
Conociendo [En lab03.doc y valley.asta]

1. En el directorio descarguen los archivos contenidos en [valley.zip](#). Revisen el código:
a) ¿Cuántos paquetes tiene? b) ¿Cuál es el propósito del paquete presentación? c) ¿Cuál es el propósito del paquete dominio?
 2. Revisen el paquete de dominio, a) ¿Cuáles son los diferentes tipos de componentes de este paquete? b) ¿Qué implica cada uno de estos tipos de componentes?
 3. Revisen el paquete de presentación, a) ¿Cuántos componentes tiene? b) ¿Cuántos métodos públicos propios (no heredados) ofrece?
 4. Para ejecutar un programa en java, ¿Qué método se debe ejecutar? ¿En qué clase se encuentra?
 5. Ejecuten el programa. ¿Qué funcionalidades ofrece? ¿Qué hace actualmente? ¿Por qué?
1. Hay dos paquetes que son : Dominio y Presentación, El propósito del paquete presentación es el de manejar la interfaz gráfica del usuario, es decir, la capa de presentación donde el usuario puede interactuar con el sistema. El propósito del paquete Domain es el de mostrar y tener la lógica, es decir la de tener los objetos principales y sus respectivas relaciones.



2. Tiene 3 tipos de componentes que son las clases de java, los archivos txt y el package de bluej. Estos implican que las clases java nos dicen las definiciones de los objetos del dominio los cuales son: Animal, Mammal, Wolf, Unit, Valley. Donde se contienen la lógica de cada uno . El package nos muestra la configuración del paquete dentro de bluej, pues nos muestra como se ven las clases en diagrama con sus relaciones, colores y tipos.
3. En el paquete de presentación se encuentran dos componentes principales que son la clase ValleyGUI y la clase de PhotoValley que se encuentra adentro de esta clase anterior. Se ofrecen un total de 4 metodos publicos que son : public Valley gettheValley, public static void main, public void paintComponent y public PhotoValley(ValleyGUI gui).
4. Para ejecutar un programa en java se debe correr el main, el cual es la primera entrada que se tiene y es obligatoria que para este caso en específico es en ValleyGUI.
5. Al ejecutar el programa, se abre una ventana que tiene una cuadrícula y se llama esta ventana SchellingValley el cual como ya se dice tiene una cuadrícula y tiene un botón en

la parte inferior que dice TIC-TAC, que al presionarlo lo que hace es actualizar el valle y repintar la cuadrícula, por lo que esta es su función principal, la de mostrar el tablero con la cuadrícula.



Arquitectura general. [En lab03.doc y valley.asta]

1. Consulte el significado de las palabras **package** e **import** de java. ¿Qué es un paquete? ¿Para qué sirve? ¿Para qué se importa? Explique su uso en este programa.

En java un paquete (package) es una forma en la que se organizan las clases e interfaces dentro de un proyecto que se realiza, por lo que funciona como una especie de carpeta en el sistema de los archivos donde se están guardando las clases. Su función es básicamente el de organización puesto que se divide el proyecto en módulos lógicos y así se pueden evitar posibles conflictos de nombres. También se usa para tener el control de acceso que pueden ser Protected y Default.

Cuando se importa un paquete se hace con el fin de hacer uso de las clases o interfaces presentes en este paquete importado que no se encuentran en el paquete actual, y así evitar tener que hacer una escritura de la ruta completa al momento de tener que usarla.

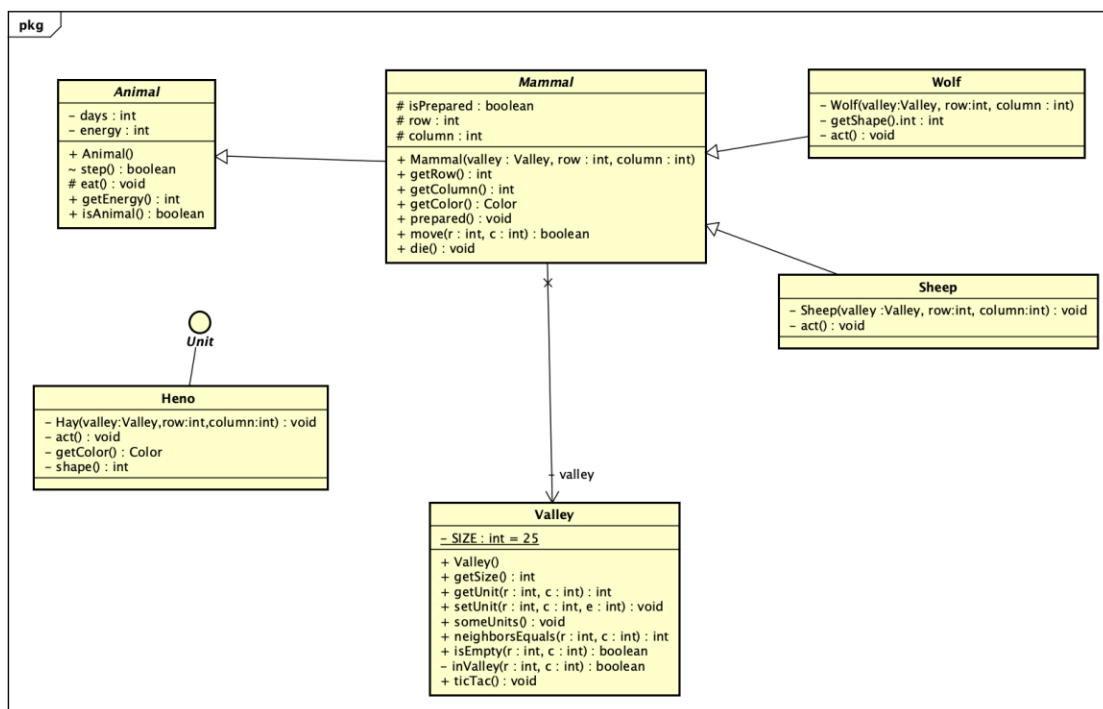
En el caso de este proyecto se usa en varias partes por ejemplo en Presentation se hace un import de Domain para poder usar dentro de Presentation las clases y demas cosas que se encuentran dentro de Domain sin hacer una escritura de ruta para usarlo.

2. Revise el contenido del directorio de trabajo y sus subdirectorios. Describa su contenido. ¿Qué coincidencia hay entre paquetes y directorios?

Al revisar el contenido se encuentra que se tiene un diseño realizado en el diagramador astah y un paquete de bluej donde está todo el conjunto de clases y paquetes del Proyecto valley. La coincidencia que hay entre paquetes y directorios es una interacción directa puesto que cada paquete corresponde a un Directorio del Sistema de archivos que se tiene por lo cual la coincidencia es de 1 a 1. Donde por ejemplo las clases que se declaran en el paquete Domain se ubican físicamente en la carpeta: /valley/domain, y pasa lo mismo en la de Presentation. Esto facilita la claridad de uso y de entendimiento.

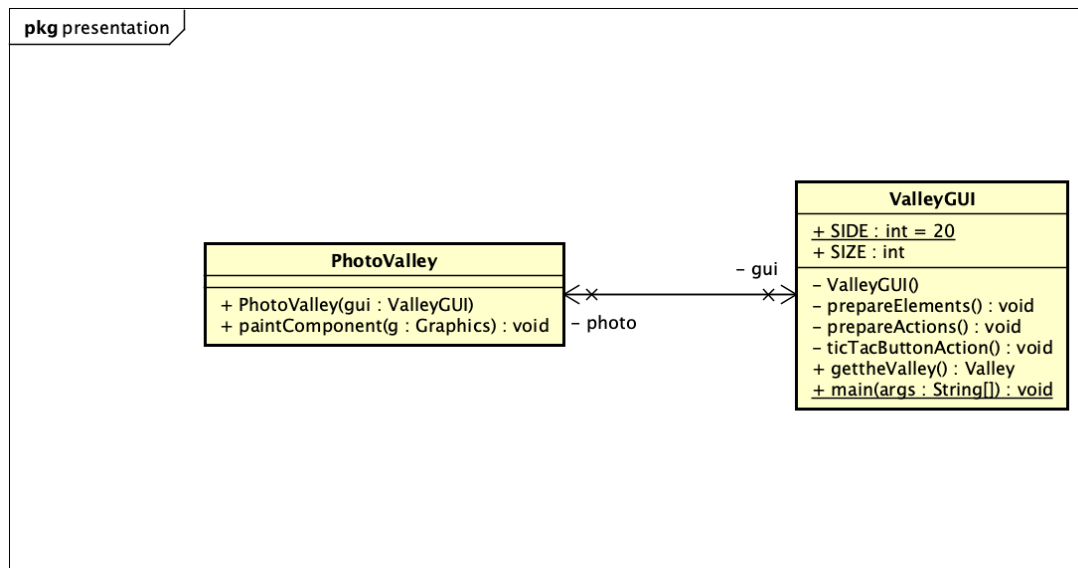
Arquitectura detallada. [En lab03.doc y valley.asta]

1. Para preparar el proyecto para **BDD**. Completen el diseño detallado del paquete de dominio. Adicionen el diagrama de clases en el paquete correspondiente. a) ¿Qué componentes hacían falta?



Componentes hacían falta:

2. Completen el diseño detallado del paquete de presentación. Adicionen el diagrama de clases al paquete correspondiente. a) ¿Por qué hay dos clases y un archivo .java?



Existen dos clases donde ValleyGUI actua como la clase principal encargada de controlar la Interfaz gráfica y la conexión con el dominio (Valley) mientras que PhotoValley es la encargada de la representación visual del entorno. Estas dos aparecen dentro de un mismo archive.java porque estan directamente relacionados entre si y estan en un mismo paquete como se miró arriba.

- Adicione la clase de pruebas unitarias necesaria para **BDD** en un paquete independiente de test. (No lo adicione al diagrama de clases) a) ¿Qué clase debe importar? b) ¿Por qué?

Ciclo 1. Iniciando con los lobos [En lab03.doc y *.java]

(NO OLVIDE BDD - MDD)

- Estudie la clase **Valley** ¿Qué tipo de colección usa para albergar cosas? ¿Puede recibir lobos? ¿Por qué?
- Estudie el código asociado a la clase **Wolf** a) ¿de qué colores es? b) ¿qué forma usa?

El tipo de colección que se usa en esta clase Valley al hacer la revision es una colección de tipo de arreglo bidimensional de la forma (Unit [] []) puesto que se usa así para representar el espacio donde se ubican cada uno de los diferentes elementos del valle. Cada ubicación de la matriz

puede obtener una herencia de la clase Unit o por el contrario ser nula si no existe un element en dicha posición de la matriz.

La clase valley si puede recibir lobos puesto que como se mencionó la matriz places está compuesta por los elementos de Unit y la clase Wolf hereda de la clase Mammal y es extendido en Animal, gracias al polimorfismo y la herencia el objeto Wolf es reconocido dentro Unit y así puede ser almacenado en la matriz sin problema.

```
package domain;
import java.util.*;

/*No olviden adicionar la documentacion*/
public class Valley{
    static private int SIZE=25;
    private Unit[][] places;

    public Valley() {
        places=new Unit[SIZE][SIZE];
        for (int r=0;r<SIZE;r++){
            for (int c=0;c<SIZE;c++){
                places[r][c]=null;
            }
        }
        someUnits();
    }
}
```

2. Estudie el código asociado a la clase [Wolf](#), a) ¿de qué color es? b) ¿qué forma usa para pintarse? c) ¿cuándo pierde su energía? d) ¿qué componentes definen la clase [Wolf](#)? Justifique sus respuestas.

```

1
2 package domain;
3 import java.awt.Color;
4
5 public class Wolf extends Mammal{
6
7     public Wolf(Valley valley,int row, int column){
8         super(valley,row, column);
9         color=Color.black;
10    }
11
12    public int getShape(){
13        return Unit.ROUND;
14    }
15
16    public void act(){
17        if (getEnergy()==0){
18            die();
19        }else{
20            if (! move(row+ (int)(Math.random() * 3) - 1,column+ (int)(Math.random() * 3) - 1)){
21                move(row+ (int)(Math.random() * 3) - 1,column+ (int)(Math.random() * 3) - 1);
22            }
23        }
24    }
25 }

```

Al revisar el código se puede observar que el color del lobo es negro pues así se declara en el constructor con la instrucción: (color = Color.black;), el lobo se pinta con la forma redonda pues en la parte de (Unit.ROUND) es una constante que se define en Unit y pues se puede asumir que se relaciona con una forma redonda. El lobo pierde su energía al moverse puesto que el movimiento implica gasto de energía que se hereda de la clase Mammal. Por medio del método Act() se llama a die() donde si no hay energía el lobo se elimina del valle por completo. Los componentes que definen la clase son la de en primer lugar el constructor, dos métodos getShape() y act(), los cuales definen el comportamiento del lobo y su representación o apariencia visual.

3. **Wolf** por ser un **Mammal**, ¿qué atributos tiene? ¿qué puede hacer (métodos)? ¿qué decide hacer distinto (sobrescritura)? ¿qué no puede hacer distinto (métodos finales)? ¿qué debe aprender a hacer (métodos abstractos)? Justifique sus respuestas.
 - a) Wolf al ser un mammal, y este al heredar de Animal, posee los atributos valley, que como se menciona en el punto de arriba representa el entorno donde habita, tiene una row y una column que son los que van a indicar la posición dentro del valle, color que va relacionado con lo visual y finalmente hereda los atributos de la clase Animal de vitalidad y energía
 - b) Esta clase al heredar de Mammal y Animal, es capaz de ejecutar métodos públicos y privados como getRow(), getColumn(), move(), getEnergy(), setEnergy() y die() y getColor(), que van a permitir que el lobo pueda desplazarse, conocer su ubicación, tener color e interactuar con su entorno.
 - c) Wolf decide hacer distinto mediante la sobrescritura de los métodos act() y getShape(), para hacer la definición de su propio comportamiento, por ejemplo el lobo se mueve de manera aleatoria dentro de valle muere al quedarse sin energía con el die() y para

representarse de manera redonda con el `getShape()`. Estos metodos se diferencian del comportamiento de las clases abstractas `Animal` y `Mammal`.

- d) Lo que no puede hacer distinto es la sobrescritura de los métodos `getRow()` y `getColumn()` puesto que al revisarlos en la clase `mammal` se ve que están definidos como `Final`, lo cual nos hace entender que el lobo tiene una forma en la que obtiene su posición es fija y no se puede alterar.
 - e) Al `Mammal` ser una clase abstracta, deja metodos para que sean implementados por su subclase, por lo cual `Wolf` tiene que aprender a definir los metodos que se mencionaron arriba de `act` y `getShape` con su propia lógica para que se diferencien de una clase clase abstracta y pueda instanciarse.
4. Por comportarse como un `Unit`, ¿qué sabe hacer? ¿qué decide hacer distinto? ¿qué no puede hacer distinto? ¿qué debe aprender a hacer? Justifique sus respuestas.
- a) Al ser un `Unit`, el lobo tiene la capacidad de comportarse como una unidad dentro el valle en si, gracias a los metodos que tiene como `shape()`, `getColor()`, `isAnimal()`, `isResource()`, gracias a todo esto el lobo sabe como comportarse y actuar e interactuar en el entorno.
 - b) Como se mencionó arriba lo que decide hacer distinto el lobo es en la parte de los metodos de `act()`, donde implementa su propia acción que como se detalló es en como moverse de forma aleatoria y en morir cuando su energía llega a 0, Tambien con el metodo de `getShape()`, para dibujarse especificamente como quiere por medio de la sobrescritura de este metodo y pintarse de forma circular y al definir su propio color en el constructor tambien hace esto distinto.
 - c) Al `Unit` ser una interfaz no cuenta con métodos `Final`, sino que cuenta con metodos `Default` y estos son heredados por `Wolf`, el cual no los altera y hereda estos metodos por defecto.
 - d) `Wolf` debe aprender a actuar de forma especifica, ya que cada tipo de `Unit` debe saber comportarse y cada uno se comporta de una forma distinta.

5. De acuerdo a lo anterior un `Wolf`, ¿Cómo actúa?

De acuerdo con todo lo que se revisó de Código, relación con demás clase, revision de atributos se puede decir que el `Wolf` se comporta de la forma:

En primer lugar el lobo tiene la capacidad de verificar su energía antes de moverse, donde revisa si es igual a 0, donde si lo es, muere y se elimina del entorno, lo cual nos da a entender que el lobo depende de su energía para la supervivencia. Si ya revisa que si tiene

energía cuenta con un posicionamiento aleatorio dentro de Valley para empezar a moverse, donde cuenta con una Row (Fila) y Column (Columna) con rangos específicos para poder tener la capacidad de moverse alrededor de estos rangos. Usa el método Move() que se hereda de sus superclases donde si cuenta con energía permanece en constante movimiento. Y finalmente se sabe que el movimiento implica consume de energía constante por lo cual consideramos cuenta con un total de 3 comportamientos directos y uno indirecto que es el ultimo que se mencionó.

6. Ahora vamos a crear dos lobos en diferentes posiciones (10,10) (15,15) , llámelos **akela y larka** , usando el método **someUnits()** . Ejecuten el programa, ¿Qué pasa con los lobos? ¿Por qué? Capturen una pantalla significativa.

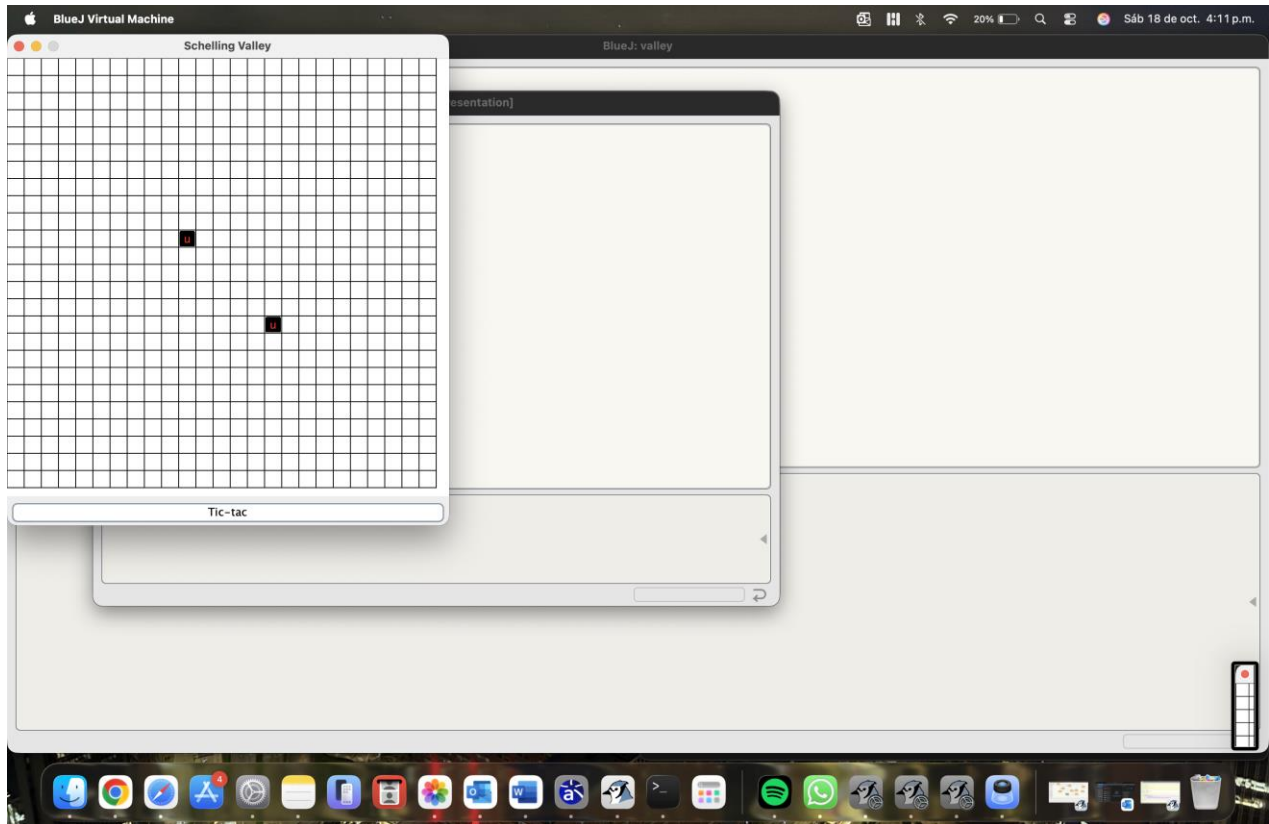
Al modificar el método de someUnits() en valley, creamos los lobos con los nombres y las posiciones que nos piden y quedan de la forma:

```
public void setUnit(int r, int c, Unit e){
    places[r][c]=e;
}

public void someUnits() {
    Wolf akela = new Wolf(this, 10, 10);
    Wolf larka = new Wolf(this, 15, 15);
    setUnit(10, 10, akela);
    setUnit(15, 15, larka);
}
```

Al ejecutar el programa desde la clase ValleyGUI, corriendo el método main nos aparece la Ventana emergente con la cuadrícula y nos muestra que los lobos fueron creados de

forma exitosa en las posiciones asignadas en la clase Valley



7. Diseñen, construyan y prueben el método llamado `ticTac()` de la clase `Valley`.

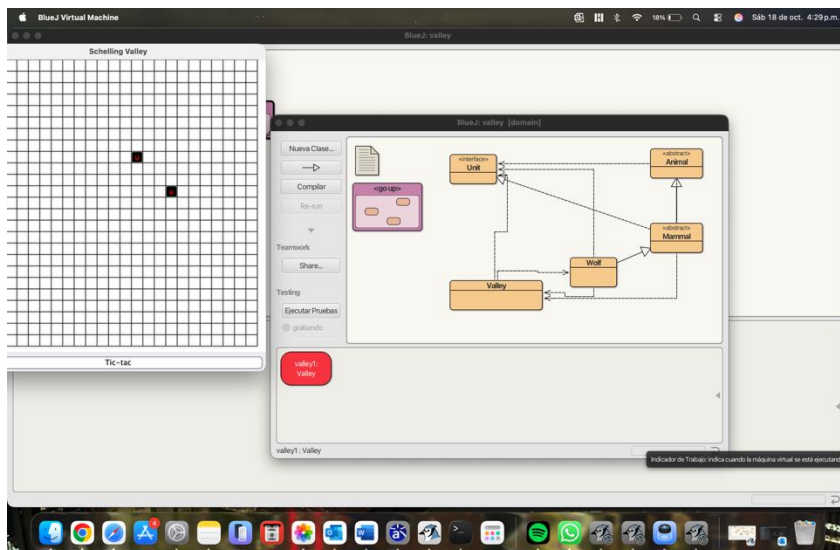
Al diseñar y construir el método `tictac` dentro de la clase `valley` queda de la forma:

```

public void ticTac(){
    //Recorre todas las filas
    for (int r = 0; r < SIZE; r++) {
        //Recorre todas las columnas
        for (int c = 0; c < SIZE; c++) {
            Unit unit = places[r][c];
            // Verificar posición si hay algo ya en esa
            if (unit != null) {
                unit.act();
            }
        }
    }
}

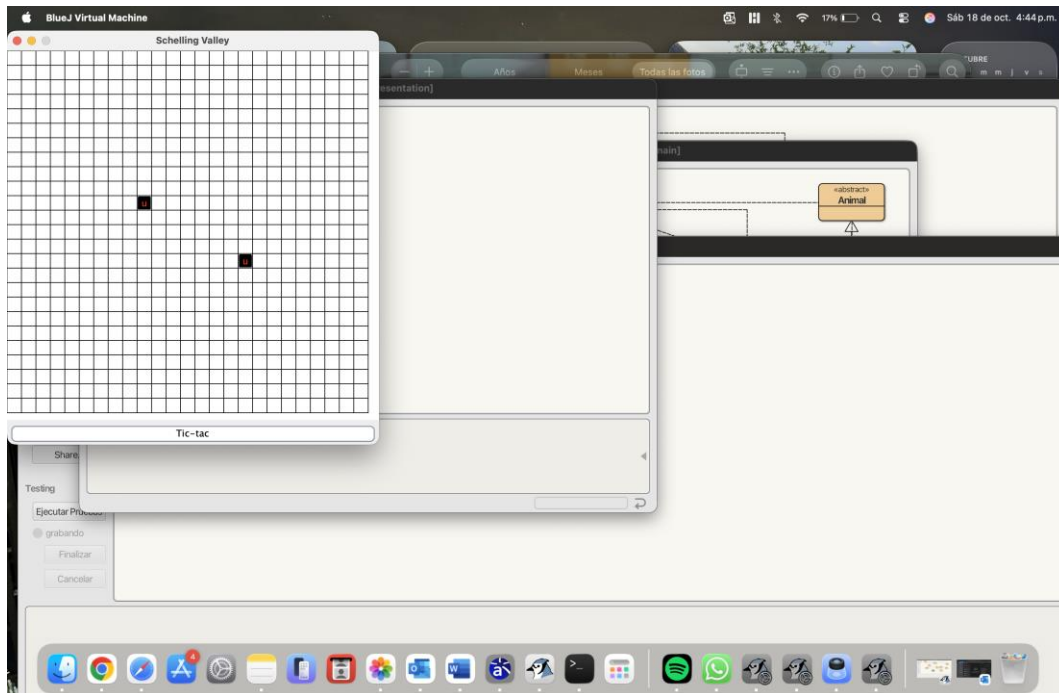
```

Para probarlo y verlo de forma visual se tiene abierta la Ventana de ValleyGUI con los lobos creados anteriormente y se presiona el boton tic-tac de abajo varias veces y se observa el movimiento que es lo esperado para este boton, un ejemplo: se tiene en esa posicion

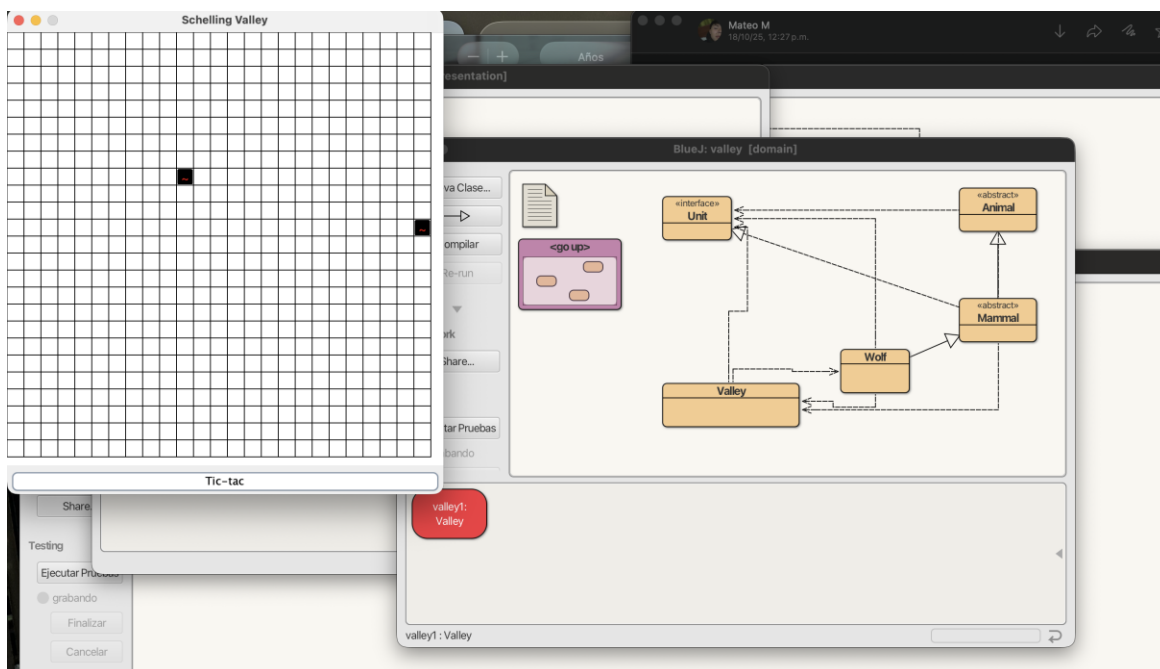


8. ¿Cómo quedarían **akela** y **larka** después de uno, sesenta y ciento uno **Tic-tac**? Ejecuten el programa. Capturen pantallas significativas en los momentos correspondientes. ¿Es correcto?

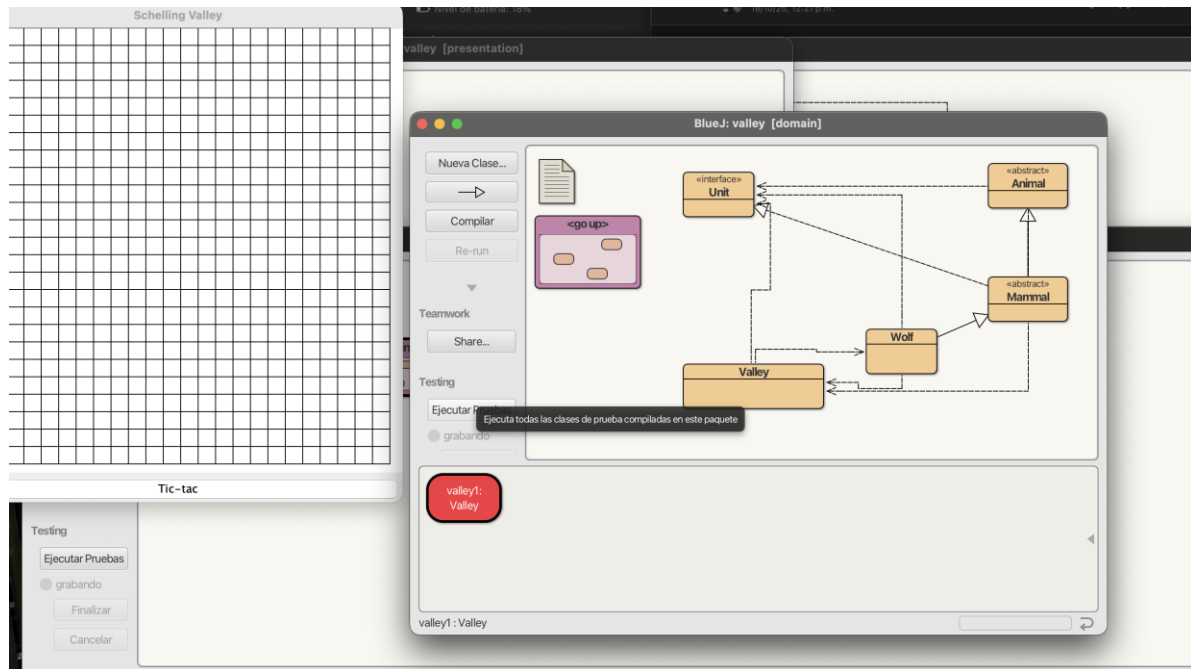
Ahora presionamos el boton de TicTac despues de darle un click se observa el movimiento:



Despues de 60 clicks aparece el simbolo ~ que significa que tiene la energía por debajo de 50 y tiene sentido teniendo en cuenta que al revisar la logica cada movimiento consume energia y al darle 60 clicks ya esta por denajo de la mitad.



Y nos damos cuenta que al presionarlo por muchas veces (cerca de 100 como pedia, se eliminaron del “tablero” donde asumimos que es porque su energía cayó a 0 y se murieron.



Ciclo 2. Incluyendo a las ovejas [En lab03.doc y valley.asta]

(NO OLVIDE BDD - MDD)

El objetivo de este punto es permitir recibir ovejas. Ellas (i) son rectangulos de color gris claro (ii) inician con 5 puntos de energía; (iii) se mueven en siempre hacia el norte y si se llegan al norte cambian de dirección camino hacia el sur (iii) cada paso pierden 1 punto de energía (iv) al llegar al norte recuperan toda su energía (v) si quedan vecinas a una oveja ganan 1 punto de energia (vi) si quedan vecinas a un lobo mueren.

1. Para implementar este nuevo **Mammal** ¿cuáles métodos se sobre-escriben (overriding)?
2. Diseñen, construyan y prueben esta nueva clase. (Mínimo dos pruebas de unidad)
3. Adicione una pareja de ovejas, llámelas **shaun** y **woolly**, (a) ¿Cómo quedarían después de tres **Tic-tac**? Ejecuten el programa y hagan tres clics en el botón. Capturen una pantalla significativa. (b) ¿Es correcto?

1. Primero se hizo un ajuste en los constructores de la clase **Animal**, para que el parámetro **energy** y **days** fuera pedido siempre y en la clase **Mammal**, se definio como constante para que **wolf** lo usara como estaba en un inicio, **energy=100**, **days=0**, Pero que cuando fuera usada por Oveja se pudiera cambiar el valor de **energy=5**.

```

public class Oveja extends Mammal {
    private static final int INITIAL_ENERGY = 5;
    private static final int INITIAL_DAYS = 0;
    private boolean movingNorth = true;

    public Oveja(Valley valley, int row, int column) {
        super(valley, row, column, INITIAL_ENERGY, INITIAL_DAYS);
        color = Color.LIGHT_GRAY;
        movingNorth = true;
    }
}

```

2.

```

package domain;

import static org.junit.Assert.*;
import org.junit.Before;
import org.junit.Test;
import java.awt.Color;

public class TestOveja {

    private Valley valley;

    @Before
    public void setUp() {
        valley = new Valley(); // tamaño pequeño de valle
    }

    /**
     * Verifica que una oveja se inicializa con color gris claro
     */
    @Test
    public void testColorInicial() {
        Oveja o = new Oveja(valley, 2, 2);
        assertEquals("El color inicial debe ser gris claro", Color.LIGHT_GRAY, o.getColor());
    }

    /**
     * Verifica que la oveja comienza con energía inicial 5
     */
    @Test
    public void testEnergiaInicial() {
        Oveja o = new Oveja(valley, 1, 1);
        assertEquals("La energía inicial debe ser 5", 5, o.getEnergy());
    }
}

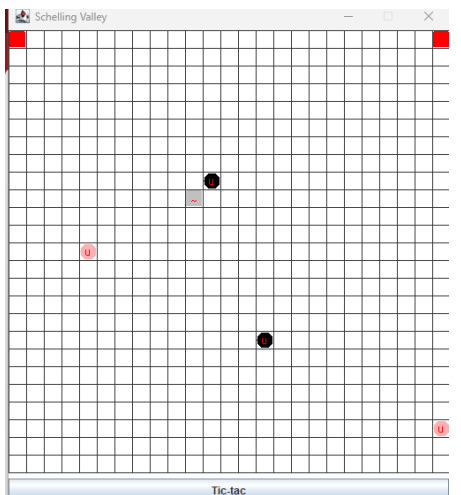
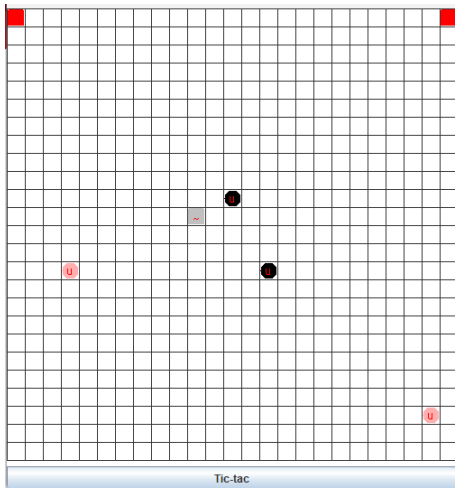
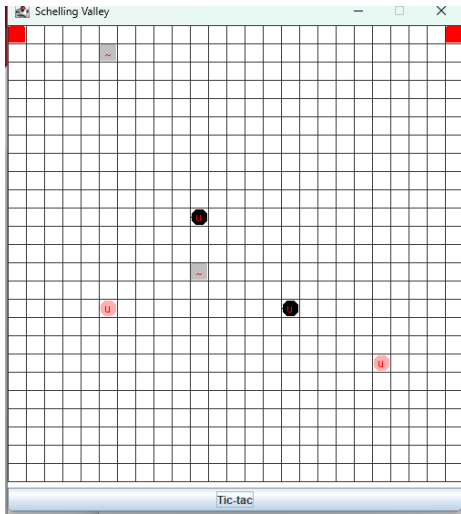
```

```

Oveja chaun = new Oveja(this, 1, 5);
Oveja woolly = new Oveja(this, 13, 10);

```

3.



Ciclo 3. Adicionando heno [En lab03.doc, valley.asta y *.java]

El objetivo de este punto es incluir paquetes de heno (sólo vamos a permitir el tipo básico de paquetes de heno) los paquetes de heno son cuadrados y van cambiando de color: rojo, amarillo, rojo, amarillo, rojo.

(NO OLVIDE BDD - MDD)

1. Para poder adicionar paquetes de heno, ¿debe cambiar en el código de `Valley` en algo? ¿por qué?

Al revisar la clase `Valley` y teniendo en cuenta que vamos a adicionar los paquetes de heno pensamos que no es necesario modificar nada de la clase `valley` puesto que esta clase puede albergar cualquier objeto de la implementación de la interfaz `UNIT` y un nuevo objeto como heno se puede adicionar sin problema. Por lo que solo es necesario crear la nueva clase de Heno y ya.

2. Diseñen , construyan y prueben esta nueva clase. (Mínimo dos pruebas de unidad)

Se diseña y construye la clase respectiva en `bluej` unida a `Unit` (ver package)

```
1 package domain;
2 import java.awt.Color;
3
4 public class Hay implements Unit{
5
6     private Valley valley;
7     private Color color;
8     private int row;
9     private int column;
10
11
12     /**
13      * Constructor del paquete de heno
14      * @param row Fila donde se ubica
15      * @param column Columna donde se ubica
16      */
17     public Hay(int row, int column){
18         this.valley = valley;
19         this.row = row;
20         this.column = column;
21         this.color = Color.red; // Iniciar con el color rojo
22     }
23
24
25     /**
26      * Define el comportamiento del heno en cada tic-tac
27      * Cambia de color alternando entre rojo y amarillo
28      * Se define la secuencia: rojo → amarillo → rojo → amarillo → rojo (enunciado)
29      */
30     @Override
31     public void act(){
32         if (color.equals(Color.red)){
33             color = Color.yellow;
34         } else {
35             color = Color.red;
36         }
37     }
38 }
39
```



```

/**
 * Retornar la forma del heno que se define como cuadrada según el enunciado
 * @return Unit.SQUARE
 */
@Override
public int shape(){
    return Unit.SQUARE;
}

/**
 * Retornar el color actual del heno
 * @return Color actual rojo o amarillo
 */
@Override
public Color getColor(){
    return color;
}

/**
 * Indica que el heno es un recurso y no un animal
 * @return true Es un recurso
 */
@Override
public boolean isResource(){
    return true;
}

/**
 * Indica que el heno no es un animal
 * @return false porque no es un animal
 */
@Override
public boolean isAnimal(){
    return false;
}

```

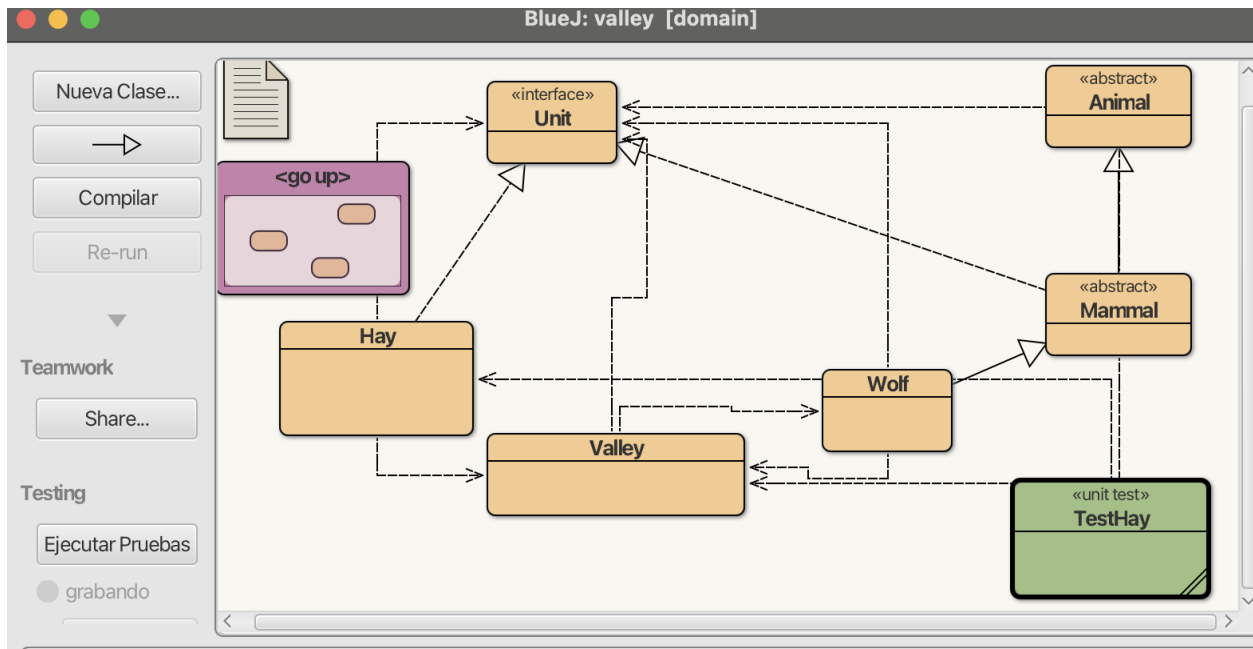
```

/**
 * Retorna la fila en la que está ubicado el heno
 * @return fila
 */
public int getRow(){
    return row;
}

/**
 * Retorna la columna donde está ubicado el heno
 * @return columna
 */
public int getColumn(){
    return column;
}

```

Las pruebas de unidad hechas son primero que el color rojo sea el primero que se tienen antes que cualquier click, y la Segunda prueba se hace para verificar que se está alternando el color con cada act() y click de Tik-Tak para ver que si esta cambiando, se crea la clase TestHay:



El código queda de la siguiente forma:

```

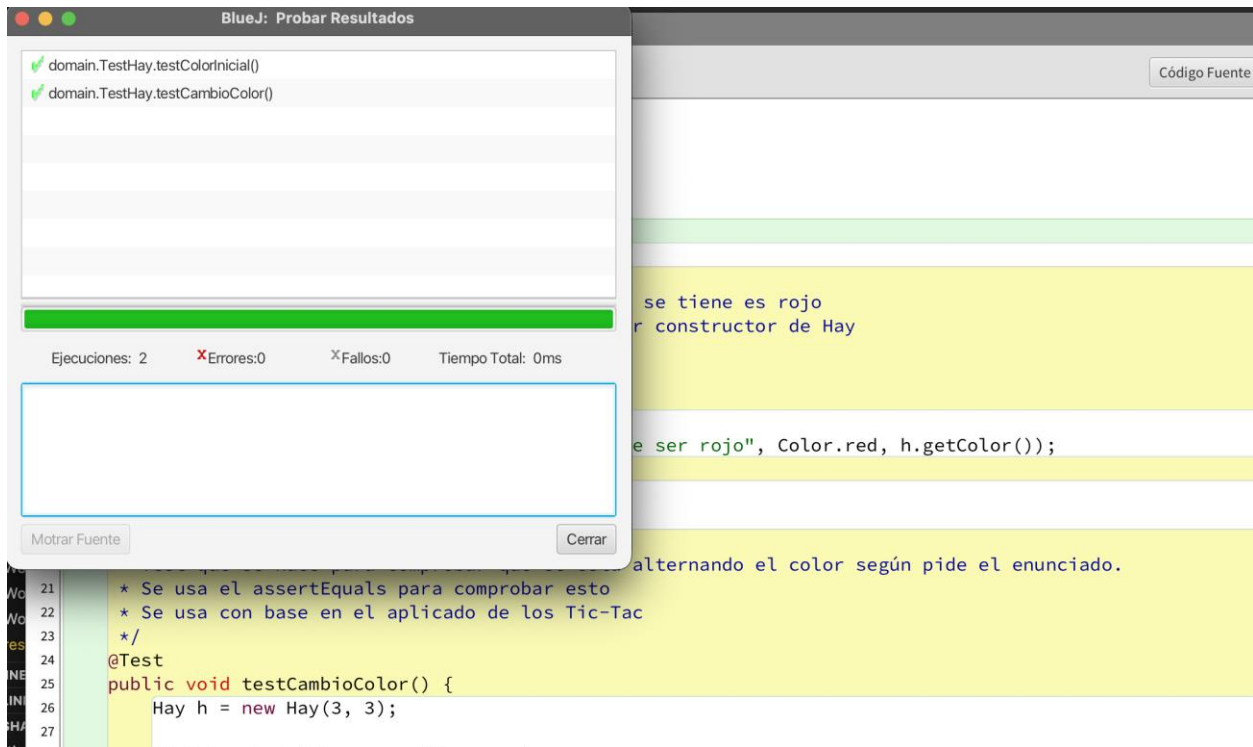
1 package domain;
2 import static org.junit.Assert.*;
3 import org.junit.Test;
4 import java.awt.Color;
5
6 public class TestHay {
7
8     /**
9      * Test para comprobar que el color inicial que se tiene es rojo
10     * Se define que el color rojo es el inicial por constructor de Hay
11     */
12     @Test
13     public void testColorInicial() {
14         Hay h = new Hay(5, 5);
15         assertEquals("El color inicial del heno debe ser rojo", Color.red, h.getColor());
16     }
17
18     /**
19     * Test que se hace para comprobar que se está alternando el color según pide el enunciado.
20     * Se usa el assertEquals para comprobar esto
21     * Se usa con base en el aplicado de los Tic-Tac
22     */
23     @Test
24     public void testCambioColor() {
25         Hay h = new Hay(3, 3);
26
27         // Color inicial que se tiene :rojo
28         assertEquals(Color.red, h.getColor());
29
30         // Primer tic-tac debe pasarse a amarillo como se define en la secuencia
31         h.act();
32         assertEquals("Después del primer click debe ser amarillo", Color.yellow, h.getColor());
33
34         // Segundo tic-tac debe volver a rojo como se define en la secuencia
35         h.act();
36         assertEquals("Después del segundo act() debe volver a rojo", Color.red, h.getColor());
37     }
38 }

```

Clase compilada - no hay errores de sintaxis

guardado

Se ejecutan las pruebas:

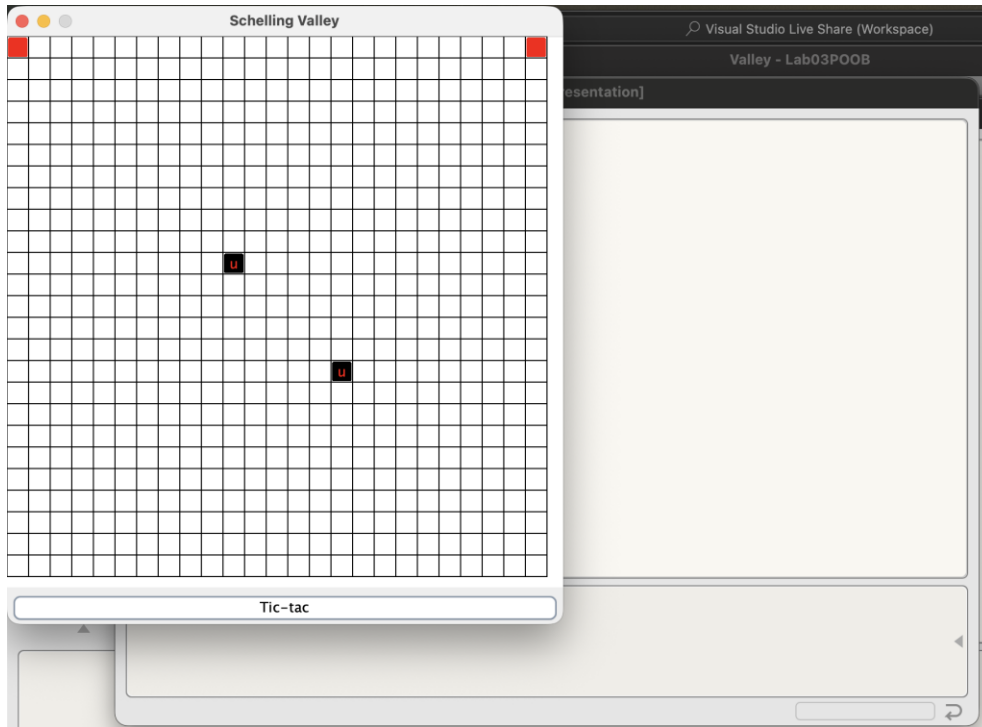


3. Adicionen dos paquetes de heno en las esquinas superiores de el valle, llámenlos `alarm` y `alert`, (a) ¿Cómo quedarían después de cuatro `Tic-tac`? Ejecuten el programa y hagan cuatro clics en el botón. Capturen una pantalla significativa. (b) ¿Es correcto?

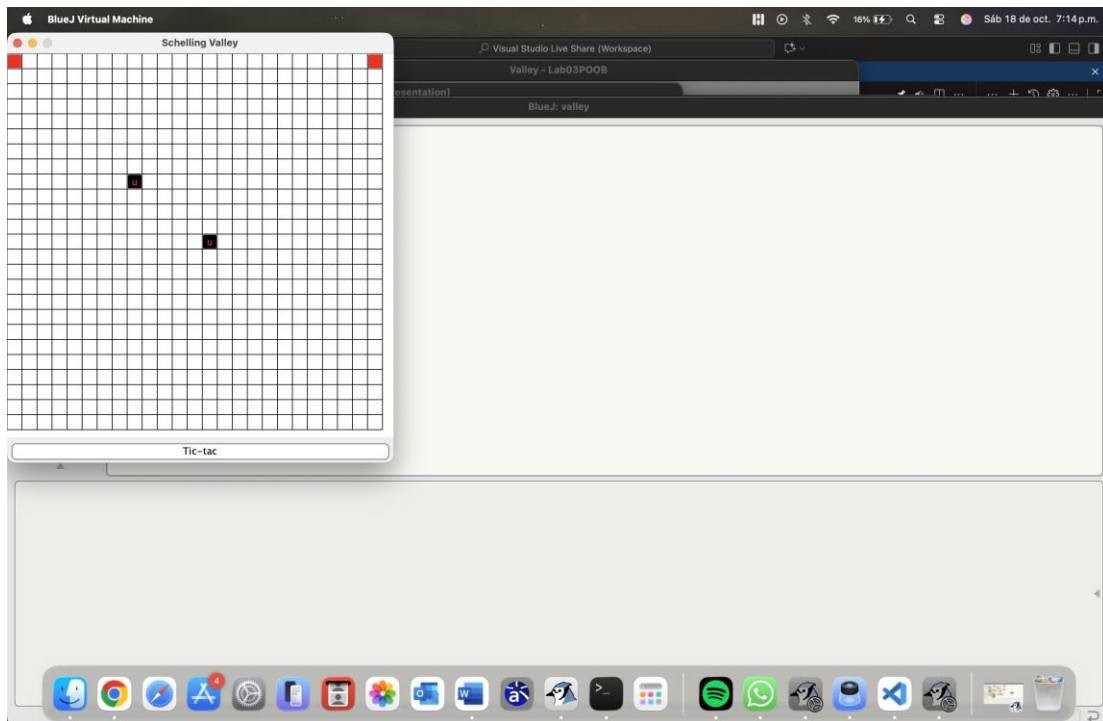
Se agregan los dos henos a la clase Valley dentro del metodo SomeUnits()

```
public void someUnits() {  
  
    Wolf akela = new Wolf(this, 10, 10);  
    Wolf larka = new Wolf(this, 15, 15);  
    Hay alarm = new Hay(0, 0);           // Esquina izquierda  
    Hay alert = new Hay(0, SIZE-1);      // Esquina derecha  
    places[0][0] = alarm;  
    places[0][SIZE-1] = alert;  
    setUnit(10, 10, akela);  
    setUnit(15, 15, larka);  
}
```

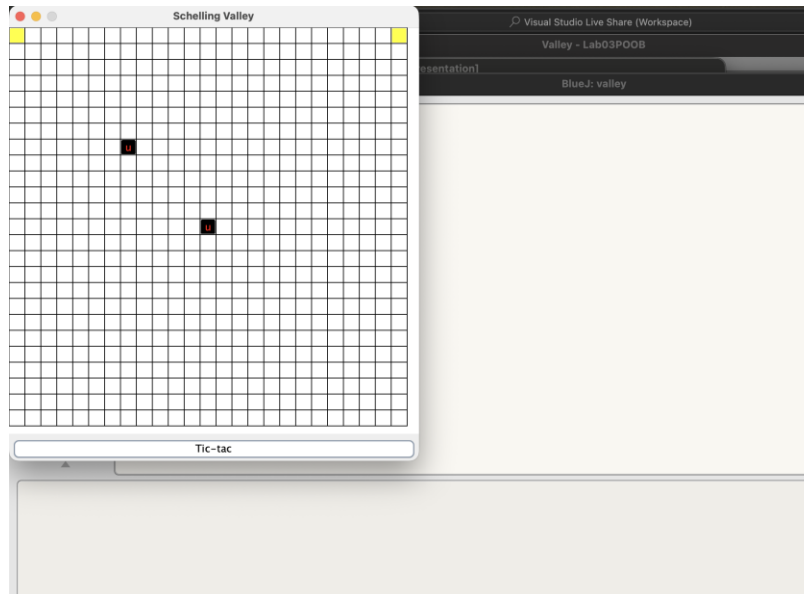
Al ejecutar la main de ValleyGUI se ve en la cuadrícula en las esquinas como se solicita.



Tras hacer cuatro clicks queda del mismo color rojo puesto que la secuencia esta en este orden y teniendo en cuenta que se inicio en rojo deberia acabar en rojo tras cuatro clicks. Por lo tanto es correcto.



Se hace otro click para ver que si cambia el color:



Ciclo 4. Nuevo Mamifero: Proponiendo y diseñando

El objetivo de este punto es permitir recibir en un nuevo tipo de Mamifero.

(NO OLVIDE BDD - MDD)

1. Propongan, describan e implementen el nuevo tipo de Mamifero.
(Mínimo dos pruebas de unidad)
 2. Considerando una pareja de ellos con el apellido de ustedes. (a) Piensen en otra prueba significativa y expliquen la intención. (b) Codifiquen la prueba de unidad correspondiente y capturen la pantalla de resultados de ejecución de la prueba. (c) Ejecuten el programa con esa prueba como prueba de aceptación y capturen las pantallas correspondientes.
-
1. El nuevo mamifero a agregar fue el Ser humano, con capacidad de matar al lobo o a la oveja. Se muestra como un triangulo rosa.
El humano pierde 1 de energía cada turno y muere si llega a 0. Cuando tiene energía completa (250) y hay máximo 1 lobo cerca, descansa. En otras situaciones, busca comida explorando las 8 celdas adyacentes con prioridades: primero intenta comer **heno** (+50 energía), luego **lobos** (+25 energía), después **ovejas** (+50 energía), y si no encuentra nada, simplemente se mueve a una celda vacía. Siempre elige aleatoriamente entre las opciones disponibles de su máxima prioridad, y su energía nunca excede los 250 puntos.

```

package domain;

import java.awt.Color;
import java.util.ArrayList;
import java.util.List;
import java.util.Random;

/**
 * Representa un humano en el valle. El humano consume energía cada tic,
 * busca y come recursos adyacentes (paja), puede matar y consumir lobos u
 * ovejas adyacentes para ganar energía, y se mueve a casillas vacías cuando
 * es necesario.
 *
 * @author MorenoRubiano
 * @version 1.0
 */
public class Humano extends Mammal {

    /** Energía máxima/inicial del humano. */
    public static final int INITIAL_ENERGY = 250;
    public static final int INITIAL_DAYS = 0;

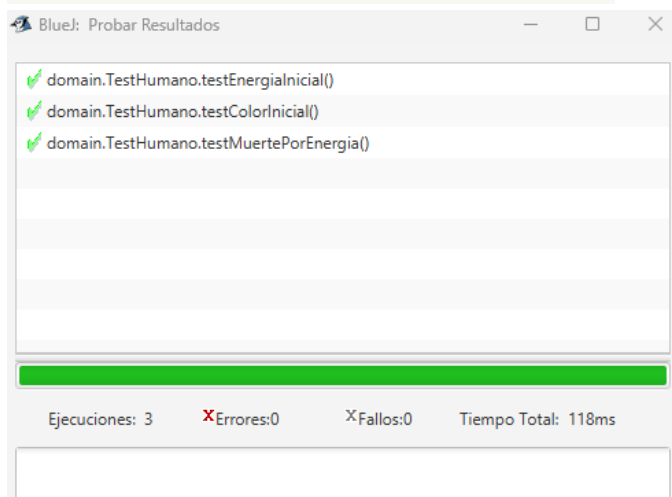
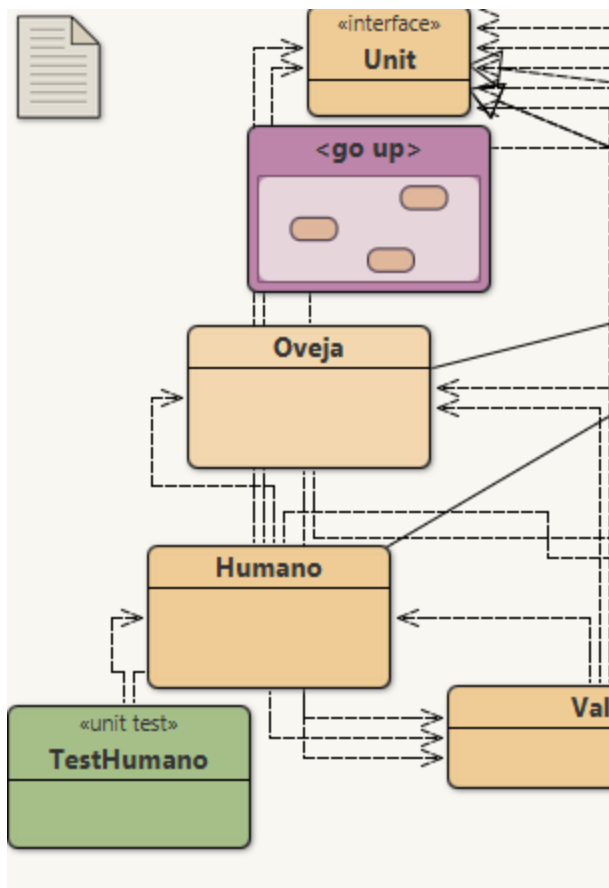
    /** Energía ganada al comer paja (Hay). */
    private static final int ENERGY_FROM_HAY = 50;

    /** Energía ganada al matar un lobo. */
    private static final int ENERGY_FROM_WOLF = 25;

    /** Energía ganada al matar una oveja. */
    private static final int ENERGY_FROM_SHEEP = 50;

```

2.



Ciclo 5. Nuevo Unit: Proponiendo y diseñando

El objetivo de este punto es permitir recibir un nuevo Unit (no Animal) en el valle **(NO OLVIDE BDD - MDD)**

1. Propongan, describan e implementen el nuevo tipo de Unit.
(Mínimo dos pruebas de unidad)
2. Considerando un par de ellos con el nombre de ustedes. (a) Piensen en otra prueba significativa y expliquen la intención. (b) Codifiquen la prueba de unidad correspondiente y capturen la pantalla de resultados de ejecución de la prueba. (c) Ejecuten el programa con esa prueba como prueba de aceptación y capturen las pantallas correspondientes.

Para el nuevo unit definimos que sea un obstaculo de la forma Hueco en la tierra que sea de color verde, Este obstaculo se define como que se queda quieto siempre y un animal al pisarlo pierde el doble de energia que causa un solo movimiento y el color del Hole se torna negro. Se implementa de la forma en bluej:

```
package domain;
import java.awt.Color;

/**
 * Clase Hole - Obstaculo Estático
 * Color : verde (color.green)
 * forma : redonda(Round)
 * Tiene posicion fija no se mueve
 * Es un recurso/obstaculo, no es un animal
 * Representa un hueco en el valle
 * @ Cristian Moreno - Juanita Rubiano
 */
public class Hole implements Unit{
    private Valley valley;
    private int row;
    private int column;
    private Color color;

    /**
     * Constructor del hole
     * @param row Fila donde se ubica
     * @param column Columna donde se ubica
     */
    public Hole(Valley valley, int row, int column) {
        this.valley = valley;
        this.row = row;
        this.column = column;
        this.color = Color.green;
        valley.setUnit(row, column, this);
    }
}
```



```

/**
 * El hole al ser obstaculo, cada vez que es pisado pierde el doble de energía
 * Cambia de color al ser pisado por algún animal
 */
@Override
public void act(){
    Unit unit = valley.getUnit(row, column);

    if (unit != null && unit instanceof Animal) {
        Animal animal = (Animal) unit;

        int energiaActual = animal.getEnergy();
        energiaActual -= 2; // pierde doble energía un animal al pisar el hueco
        animal.setEnergy(energiaActual);

        // Mostrar visualmente que fue pisado cambiando de color
        color = Color.darkGray;
    } else {
        color = Color.green;
    }
}

```

```

/**
 * Retorna la forma de la del hole ( Redonda )
 * @return Unit.ROUND
 */
@Override
public int shape(){
    return Unit.ROUND;
}

```

```

/**
 * Retorna el color del hole verde
 * @return Color.gray
 */
@Override
public Color getColor(){
    return color;
}

```

```

/**
 * Indica que el hueco es un recurso/obstáculo
 * @return true
 */
@Override
public boolean isResource(){
    return true;
}

```

```

/**
 * Indica que el hole no es un animal sino catalogado como recurso
 * @return false
 */
@Override
public boolean isAnimal(){
    return false;
}

```

```

/**
 * Retorna la fila donde está el hueco
 * @return fila
 */
public int getRow(){
    return row;
}

```

Se agrega al Valley dentro del metodo someUnits con los nombres de nuestros apellidos:

```

public void someUnits() {

Wolf akela = new Wolf(this, 10, 10);
Wolf larka = new Wolf(this, 15, 15);

Hay alarm = new Hay(this, 0, 0);           // Esquina izquierda
Hay alert = new Hay(this, 0, SIZE-1);       // Esquina derecha
places[0][0] = alarm;
places[0][SIZE-1] = alert;

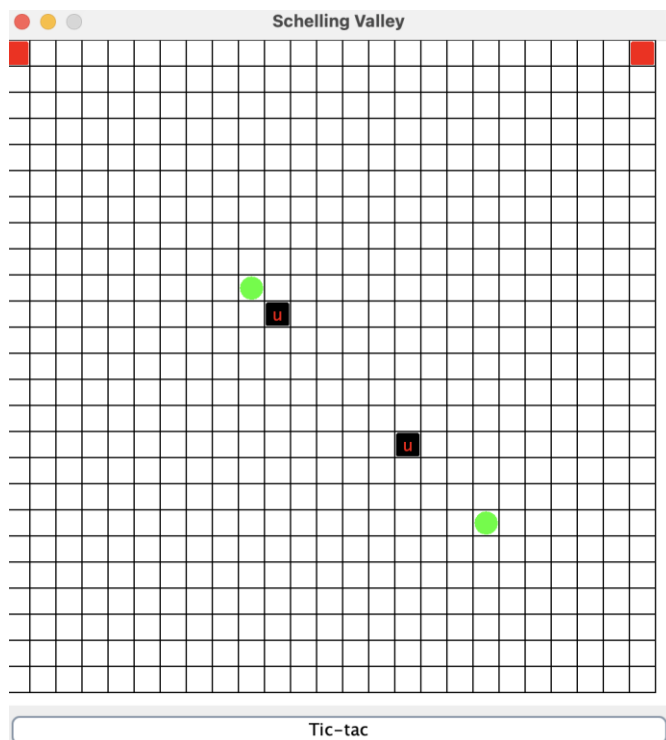
setUnit(10, 10, akela);
setUnit(15, 15, larka);

Hole Moreno = new Hole(this, 9, 9);
Hole Rubiano= new Hole(this, 18, 18);

places[9][9] = Moreno;
places[18][18] = Rubiano;
}

```

Visualmente se ven de la forma que se especifico con el color que se determinó y se ejecuta el programa:



Las pruebas que se hicieron de unidad es que primero al un animal pisar el hole pierda el doble de su energía y la Segunda verifica que se inicializa bien el hole con su color, tipo de recurso o animal entre otros:

```
package domain;

import static org.junit.Assert.*;
import org.junit.Test;
import java.awt.Color;

/**
 * Pruebas unitarias para la clase Hole.
 * Autores: Cristian Moreno / Juanita Rubiano
 */
public class TestHole {

    /**
     * Verifica que el Hole se inicialice correctamente:
     * - Color verde
     * - Forma redonda
     * - Sea un recurso, no un animal
     * - Se registre en el Valley en la posición correspondiente
     */
    @Test
    public void testInicializacionHole() {
        Valley valley = new Valley();
        Hole hole = new Hole(valley, 5, 5);

        assertEquals("El color inicial del hole debe ser verde", Color.green, hole.getColor());
        assertEquals("El hole debe tener forma redonda", Unit.ROUND, hole.shape());
        assertTrue("El hole debe ser un recurso", hole.isResource());
        assertFalse("El hole no debe ser un animal", hole.isAnimal());
        assertEquals("El hole debe estar en la posición (5,5)", hole, valley.getUnit(5, 5));
    }
}
```

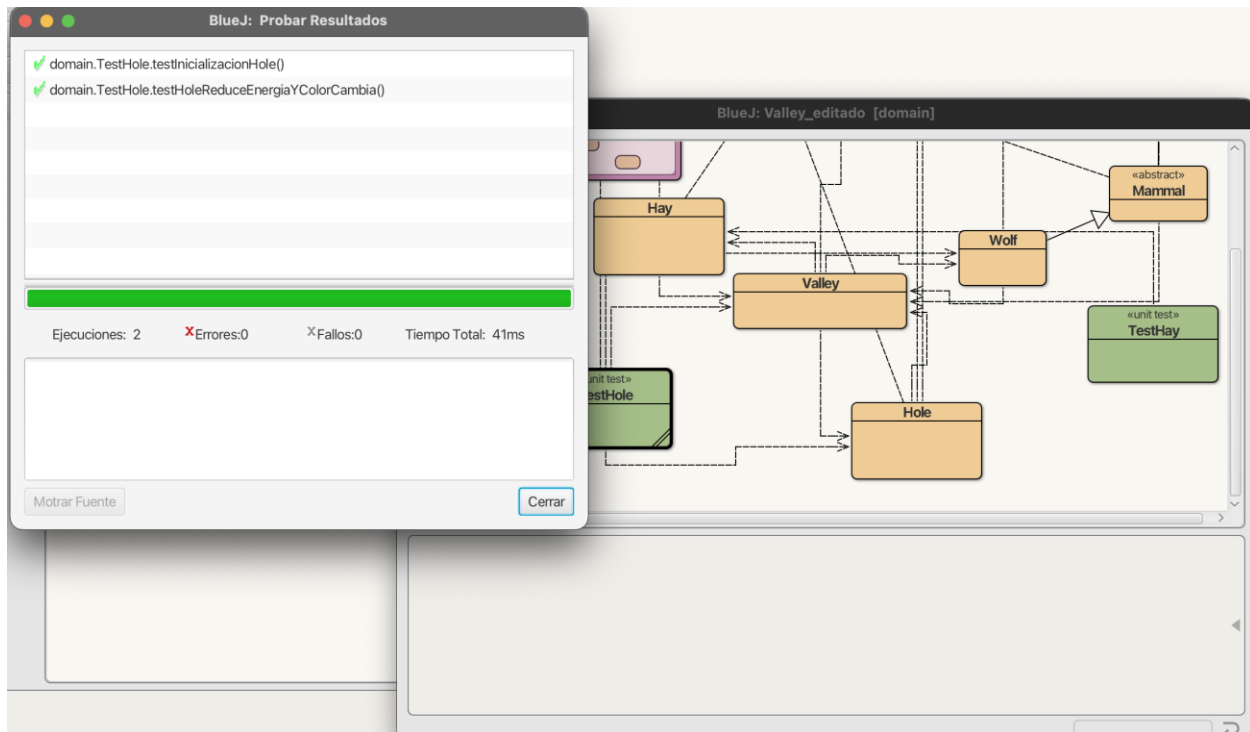
```
/**
 * Verifica que al actuar, el hole haga que un animal pierda el doble de energía
 * y cambie su color a gris oscuro.
 */
@Test
public void testHoleReduceEnergiaYColorCambia() {
    Valley valley = new Valley();
    Hole hole = new Hole(valley, 10, 10);

    // Crear un lobo y lo colocamos sobre el mismo espacio del hole
    Wolf wolf = new Wolf(valley, 10, 10);
    wolf.setEnergy(50);
    valley.setUnit(10, 10, wolf);

    // Ejecutar act() del hole
    hole.act();

    assertEquals("El color debe cambiar a gris oscuro al ser pisado", Color.darkGray, hole.getColor());
    assertEquals("El lobo debe perder 2 unidades de energía", 48, wolf.getEnergy());
}
}
```

Al probarlas las dos en bluej:



DE BLUEJ A CONSOLA

En esta sección del laboratorio vamos a aprender a usar java desde consola. Para esto se va a trabajar con el proyecto del punto anterior.

Comandos básicos del sistema operativo [En lab03.doc]

es de iniciar debemos repasar los comandos básicos del manejo de la consola.

1. Investiguen los comandos para moverse en la estructura de directorios: crear, borrar, listar su contenido y copiar o eliminar un archivo.
2. Organicen un nuevo directorio con la estructura propuesta para probar desde allí su habilidad con los comandos de consola. Consulten y capturen el contenido de su directorio

```
lotVol
  src
    domain
    presentation
    test
```

3. En el directorio copien únicamente los archivos *.java del paquete de aplicación . Consulte y capture el contenido de `src/domain`

Comandos para moverse en directorios:

1. `cd` - cambiar de directorio
2. `mkdir` - crear directorio
3. `rm` o `rmdir` - borrar directorio
4. `ls` o `dir` - listar contenido
5. `cp` - copiar archivos/directorios

6. mv - mover o renombrar

2.

```
C:\Users\JuanitaRubiano\Documents\lotVol
|
└── src
    ├── domain
    ├── presentation
    └── test
```

Para comprobar que las carpetas se habían creado correctamente, usó el comando:

```
tree C:\Users\JuanitaRubiano\Documents\lotVol /f
```

El resultado mostrado en consola fue:

```
C:\USERS\JUANITARUBIANO\DOCUMENTS\LOTVOL
└── src
    ├── domain
    ├── presentation
    └── test
```

3.

Luego, se copiaron únicamente los archivos .java del paquete domain del proyecto original al nuevo directorio src/domain:

Comando usado:

```
copy C:\ProyectoOriginal\domain\*.java
C:\Users\JuanPablo\Documents\lotVol\src\domain\
```

Después de copiar, el contenido del directorio src/domain se verificó con:

```
dir C:\Users\JuanitaRubiano\Documents\lotVol\src\domain
```

Y el resultado obtenido fue:

```
C:\USERS\JUANITA\DOCUMENTS\LOTVOL\SRC\DOMAIN
|   Animal.java
|   Oveja.java
|   Wolf.java
```

```
| Hay.java
| Hole.java
| Humano.java
| Valley.java
| Unit.java
| Mammal.java
```

O usando tree:

```
C:\USERS\JUANITA\DOCUMENTS\LOTVOL\SRC
```

```
└─domain
    Animal.java
    Hay.java
    Hole.java
    Humano.java
    Mammal.java
    Oveja.java
    Unit.java
    Valley.java
    Wolf.java
```

-
3. En el directorio copien únicamente los archivos *.java del paquete de aplicación . Consulte y capture el contenido de `src/domain`

Estructura de proyectos java [En lab03.doc]

En java los proyectos se estructuran considerando tres directorios básicos.

```
lotVol
  src
  bin
  docs
```

1. Investiguen los archivos que deben quedar en cada una de esas carpetas y la organización interna de cada una de ellas.
2. ¿Qué archivos debería copiar del proyecto original al directorio bin? ¿Por qué? Cópielos y consulte y capture el contenido del directorio que modificó.

Comandos de java [En lab03.doc]

1. Consulte para qué sirven cada uno de los siguientes comandos:

```
javac
java
javadoc
jar
```

2. Cree una sesión de consola y consulte en línea las opciones de los comandos java y javac. Capture las pantallas.
3. Busque la opción que sirve para conocer la versión a que corresponden estos dos comandos. Documente el resultado.

Estructuras de proyectos java

1.src (source):

Contiene el código fuente del programa, es decir, los archivos con extensión .java.

por ejemplo: el .dominio

bin (binary):

Contiene los archivos compilados del proyecto, es decir, los archivos .class generados a partir de los .java del directorio src.

La estructura de carpetas en bin debe ser igual a la de src, respetando los mismos paquetes.

docs (documentación):

Incluye la documentación generada automáticamente con Javadoc.

Contiene archivos HTML, scripts y recursos que describen las clases, métodos y atributos del proyecto.

Por ejemplo: index.html,

2. En una sesión de consola, se pueden consultar las opciones disponibles con los siguientes comandos:

```
java --helpjavac --help
```

Estos comandos muestran una lista de argumentos y parámetros que permiten personalizar la ejecución y compilación

Comandos de java

1

- **.javac**
Es el compilador de Java. Convierte los archivos de código fuente con extensión .java en archivos bytecode con extensión .class, que pueden ser ejecutados por la máquina virtual de Java (JVM).
- **java**
Ejecuta programas Java previamente compilados. Interpreta los archivos .class generados por javac.
- **avadoc**
Genera automáticamente documentación en formato HTML a partir de los comentarios especiales del código fuente Java.
- **jar**
Permite empaquetar varios archivos Java (por ejemplo .class, imágenes o recursos) en un solo archivo comprimido .jar (Java ARchive), facilitando la distribución de programas.

2.

```
Microsoft Windows [Versión 10.0.26100.6899]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Jnita> java
Usage: java [java options...] <application> [application arguments...]

Where <application> is one of:
  <mainclass>           to execute the main method of a compiled main class
  -jar <jarfile>.jar     to execute the main class of a JAR archive
  -m <module>[/<mainclass>] to execute the main class of a module
  <sourcefile>.java     to compile and execute a source-file program

Where key java options include:
  --class-path <class path>
    where <class path> is a list of directories and JAR archives to search for class files, separated by ";"
  --module-path <module path>
    where <module path> is a list of directories and JAR archives to search for modules, separated by ";"
  -version
    to print product version to the error stream and exit

For additional help on usage:      java --help
For an interactive Java environment: jshell

C:\Users\Jnita>
```



```

PS C:\Users\Jnita> javac
Usage: javac <options> <source files>
where possible options include:
  @<filename>           Read options and filenames from file
  -Akey[=value]         Options to pass to annotation processors
  --add-modules <module>(<module>)*
                        Root modules to resolve in addition to the initial modules,
                        or all modules on the module path if <module> is ALL-MODULE-PATH.
  --boot-class-path <path>, -bootclasspath <path>
                        Override location of bootstrap class files
  --class-path <path>, -classpath <path>, -cp <path>
                        Specify where to find user class files and annotation processors
  -d <directory>        Specify where to place generated class files
  -deprecation
                        Output source locations where deprecated APIs are used
  --enable-preview
                        Enable preview language features.
                        To be used in conjunction with either -source or --release.
  -encoding <encoding>  Specify character encoding used by source files
  -endorseddirs <dirs>  Override location of endorsed standards path
  -extdirs <dirs>       Override location of installed extensions
  -g                    Generate all debugging info
  -g:{lines,vars,source}
                        Generate only some debugging info
  -g:none               Generate no debugging info
  -h <directory>
                        Specify where to place generated native header files
  --help, -help, -?     Print this help message
  --help-extra, -X      Print help on extra options
  -implicit:{none,class}

```

3. Para conocer la versión de Java instalada y a la que pertenecen los comandos java y javac, se usa la opción:

```

java -versionjavac -versionjar cvf MiPrograma.jar *.class javadoc MiPrograma.javajava
MiProgramajavac MiPrograma.java

```

RETROSPECTIVA

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas/Hombre)
2. ¿Cuál es el estado actual de laboratorio? ¿Por qué? (Para cada método incluya su estado)
3. Considerando las prácticas XP del laboratorio de hoy ¿por qué consideran que son importante?
4. ¿Cuál consideran fue su mayor logro? ¿Por qué? ¿Cuál consideran que fue su mayor problema? ¿Qué hicieron para resolverlo?
5. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?
6. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

1. El tiempo total invertido fue:

Cristian: 18 horas

Juanita: 17 horas

2. El estado actual del laboratorio incompleto por un mal manejo del tiempo.
3. Son importantes para el diseño de un código limpio y óptimo y hoy en específico se usó bastante el de codificar pruebas antes de escribir código y se desarrolló en varios puntos del laboratorio y fue útil para que se garantice un programa esperado.
4. El mayor logro fue el de poder entender los puntos y realizarlos en lo que consideramos que los puntos que están hechos están bien realizados. El mayor problema tal vez fue la comunicación entre nosotros, pues hubo un problema de control de versiones porque nos generó conflictos, lo solucionamos editando en un mismo editor en tiempo real para evitar dichos conflictos.
5. Hicimos buscar el logro casi total del laboratorio, nos comprometimos a comunicarnos más y hacer las cosas con tiempo.
6. Las referencias usadas fueron las proporcionadas por el enunciado, ya sea el simulador, los archivos presentes en la Plataforma Moodle y en ciertos puntos para identificación de errores se usó IA pero no como copia y pega sino como identificación de errores.